

Batch 4 — Design Canvas Flow

System Design Teacher Platform • Build summary with plain-language file explanations

What was delivered

Batch 4 is the **headline feature** of the entire project. After three batches of foundation work, this is the one you point at in interviews. The user picks a system-design challenge ("Design Twitter"), drags components onto a canvas, connects them with arrows, submits, and an AI returns structured feedback grouped by severity — with the affected components literally lighting up on the canvas when you click each piece of feedback.

This is also the first batch where every uncached request **spends real money** on OpenAI. Most of the engineering effort goes into making sure that money is well spent: a layout-invariant diagram-hash cache means two users who design the same system structurally share one piece of feedback, and only one of them paid for it. Combined with rate limits (2 submissions per user per day, 100 globally), expected steady-state cost is under \$5/month for active development.

Total file changes	44 (29 new, 15 modified)
Reviewable units	23 (22 backend files + 1 frontend zip)
New ADRs	ADR-002 (Cosmos Mongo API) + ADR-005 (JSON over images)
New API endpoints	POST /attempts/design, GET /questions/design, GET /questions/design/{id}
New domain types	Diagram, DiagramNode, DiagramEdge, DesignFeedback, FeedbackItem, CachedFeedback
Frontend additions	React Flow canvas, 14-component palette, severity-grouped feedback panel
New frontend dep	@xyflow/react (~120 KB gzipped) — only new dependency
Cache TTL default	30 days (configurable)
LLM model	gpt-4o for feedback (~2,000–3,000 tokens per call, ~\$0.05–\$0.15)

The big picture, in plain language

The headline UX moment. The user drags 5 boxes onto a canvas (User, Load Balancer, Cache, Database, Queue). They connect them with arrows. They click "Submit." A loading spinner appears for 10–20 seconds with cycling progress messages. Then a panel slides in from the right: "Overall score: 5/10. Strengths: 2 things. Critical gap: missing auth. Important gap: no rate limiter." They click on the "missing auth" item. Suddenly the API Gateway and Database boxes light up with a red ring; everything else dims to 30% opacity. They can see exactly what the AI is criticizing.

Why caching matters more here than anywhere else. Each AI call costs ~\$0.05–\$0.15. Without caching, two students who both build the same standard "Design Twitter" architecture each pay the toll. With caching, the second one gets the same feedback for free. The trick is the hash function: it normalizes away cosmetic things (where you put the box on screen, the exact node IDs) but keeps the things that matter (which component types are connected to which). Two structurally identical designs map to the same key, regardless of layout.

Three-layer cost protection. Validation rejects malformed diagrams before any cost. The rate limiter caps users at 2 submissions per day and the whole system at 100. The cache saves on the second-and-later submissions of the same structure. If the AI returns malformed JSON, we retry once with stricter instructions, then fail gracefully. There is no realistic way for the bill to surprise us.

The flow, end to end

1. User drags components, connects them, types optional notes, clicks Submit
2. Frontend strips UI-only fields (positions, drag state) and sends clean JSON
3. Backend validates the diagram (no dangling edges, all node IDs unique)
4. Backend confirms the question is real and is a design question (not a situation)
5. Rate limiter consumes a slot — returns 429 if user or global cap is exhausted
6. Backend computes the canonical hash of the diagram + question ID
7. Backend checks the feedback cache for that hash
- 8a. **Cache HIT:** increment hit_count, return cached feedback (no AI call, \$0 cost)
- 8b. **Cache MISS:** send prompt + diagram JSON to OpenAI gpt-4o
9. Pydantic validates the AI's response. Retry once on validation failure
10. Store the new feedback in the cache with 30-day TTL (best-effort, errors logged)
11. Always insert an Attempt record into the user's history
12. Return 201 Created with feedback + attempt + cache_hit flag + rate-limit info
13. Frontend renders the feedback panel grouped by severity (critical / important / suggestion)
14. User clicks any feedback item — affected nodes get a colored ring; others dim

File-by-file breakdown (plain language)

Every file in this batch, in the order it was generated. Files are grouped by purpose so you can read just the parts you care about.

Group 1: Architecture Decision Records (2 files)

Short documents explaining *why* we made big technical choices. Future you (and interviewers) will thank you for these.

#1 `docs/adr/002-mongo-api-for-cosmos.md`

What it is: A document explaining why we'll use Cosmos DB's MongoDB API (instead of Cosmos's native NoSQL API) when we deploy to Azure in Batch 5.

Why it exists: The big payoff: we don't need to write a separate "Cosmos adapter" — the same MongoDB code we wrote in Batch 1 works against Cosmos with just a different connection string. This ADR captures that reasoning so it doesn't get forgotten.

#2 `docs/adr/005-json-over-image-submission.md`

What it is: Explains why the user submits a structured JSON description of their diagram, not a screenshot.

Why it exists: Three reasons: caching is trivial with structured data (you can hash it), the AI is more reliable on text than vision, and the AI can reference specific node IDs in its feedback so the canvas can highlight the right boxes. Image submission would be 10× more expensive and impossible to cache.

Group 2: Domain entities (2 files)

Pure data structures that define what a "diagram" and a "feedback" actually are. No logic, no I/O — just shape definitions with validation rules.

#3 `backend/app/core/domain/diagram.py`

What it is: Defines the shape of a submission: `DiagramNode` (a single box with a type and a label), `DiagramEdge` (a connection between two boxes), and `Diagram` (the whole thing). Includes a list of all 14 valid component types and size limits (max 200 nodes, 500 edges).

Why it exists: These types are used everywhere downstream. Pydantic enforces them: if a user sends a node with an unknown type, the request gets rejected before any AI call is made. Single source of truth for what a valid diagram looks like.

#4 `backend/app/core/domain/design_feedback.py`

What it is: Defines the shape of what the AI returns: `DesignFeedback` (overall score 1–10, list of strengths, list of gaps, missing components, trade-off questions, estimated level), `FeedbackItem` (one specific issue with severity and category), and `CachedFeedback` (the same plus cache metadata).

Why it exists: Pydantic uses these to validate the AI's response. If the AI returns garbage that doesn't match the schema, we retry once and then fail gracefully. The schema also drives what the frontend can render.

Group 3: The cache port and adapter (2 files modified)

#5 `backend/app/core/ports/database.py`

What it is: Adds `FeedbackCacheRepository` to the database port. Defines three methods: `get` (returns cached feedback or raises `NotFoundError`), `insert` (stores feedback with a TTL), `increment_hit_count` (tracks how popular each cache entry is).

Why it exists: The contract that any cache implementation must satisfy. By defining it as a port, we can swap implementations later (e.g., a Redis-backed version) without changing the calling code.

#6 [backend/app/adapters/local/mongodb_database.py](#)

What it is: Implements the cache contract using MongoDB. Creates two indexes on the `feedback_cache` collection: a unique index on the cache key (so you can't insert duplicates) and a TTL index on `expires_at` (so MongoDB automatically deletes old entries).

Why it exists: MongoDB's TTL indexes do the cleanup for us — no cron job, no application code to manage expiry. Set `expires_at` to "30 days from now" when inserting, and MongoDB sweeps it away when the time comes.

Group 4: The brain — hash service and prompt (3 files)

These three files are the heart of the cost-saving strategy. The hash function determines cache hit rate; the prompt determines feedback quality.

#7 `backend/app/core/services/diagram_hash_service.py`

What it is: About 50 lines of code that turn any diagram into a 64-character hash. Sorts nodes by type and label, sorts edges by their endpoint types and labels, drops UI fields, JSON-serializes deterministically, then SHA-256.

Why it exists: This is THE most important 50 lines in the batch. Two diagrams that look different on screen but represent the same architecture produce the same hash. That's what makes the cache useful: structural identity, not pixel identity. Without this, the cache would miss 99% of the time.

#8 `backend/app/core/prompts/__init__.py`

What it is: Empty file that marks the prompts directory as a Python package.

Why it exists: Pure plumbing. Required so we can import from the prompts directory.

#9 `backend/app/core/prompts/design_feedback.md`

What it is: The actual instructions sent to the AI. Sets up its role ("senior software engineer"), explains what good feedback looks like, defines what each severity level means, includes two few-shot examples (a terse junior submission and an over-engineered mid one), and contains an anti-injection guard that tells the AI to ignore any instructions in the user's diagram or notes.

Why it exists: This is the file that determines feedback *quality*. The few-shot examples are especially important — LLMs anchor strongly to demonstrated style. Two contrasting examples teach the model to be calibrated rather than always-positive or always-critical. Lives in Git so we can version it.

Group 5: The orchestrator (1 file)

#10 `backend/app/core/services/design_feedback_service.py`

What it is: The main service that ties everything together. Validates the diagram → looks up the question → consumes a rate-limit slot → hashes the diagram → checks the cache → calls the AI on miss (with one retry on validation failure) → stores the result in the cache → records the user's attempt → returns everything.

Why it exists: This is the heart of the batch. Every cost-protection mechanism converges here in a specific order: validate (cheap) before rate-limit (consumes quota) before cache check (saves spend) before LLM (the actual money). The order matters.

#11 `backend/app/core/services/question_service.py`

What it is: Adds one new method to the existing question service: `fetch_random_design_question`. No rate limit on this one.

Why it exists: Browsing design questions is free — the rate limit applies on submission, not on fetching the prompt. Otherwise users would burn quota just looking at questions.

Group 6: API contracts (4 files)

Pydantic models that define the JSON shapes flowing in and out of the API. The line between the public contract and the internal domain.

#12 `backend/app/api/schemas/diagrams.py`

What it is: API DTOs for diagrams: `DiagramNodeDTO`, `DiagramEdgeDTO`, `DiagramDTO`. Mirrors the domain types but lives separately.

Why it exists: Keeps the public API contract independent of internal implementation. If we ever add a private field to the domain entity (like a "trust score"), the API DTO doesn't accidentally expose it.

#13 `backend/app/api/schemas/design_feedback.py`

What it is: API DTOs for the feedback shape returned to the frontend.

Why it exists: Same separation principle as above — the AI's response shape lives behind a public contract the frontend depends on.

#14 `backend/app/api/schemas/attempts.py`

What it is: Adds the request and response shapes for design submissions: `SubmitDesignAttemptRequest` and `SubmitDesignAttemptResponse`. The response bundles four things: the saved attempt, the feedback, a `cache_hit` boolean, and current rate-limit usage.

Why it exists: One round-trip gives the frontend everything it needs to render: the feedback (the headline), whether this was a cache hit (small UI hint), and how many submissions are left today (banner).

#15 `backend/app/api/schemas/questions.py`

What it is: Adds `DesignQuestionResponse` as a marker class.

Why it exists: Today it's identical to `QuestionResponse`, but separating them now lets us add design-only fields later without affecting situation responses.

Group 7: HTTP endpoints (2 files modified)

#16 `backend/app/api/routes/questions.py`

What it is: Adds two endpoints: `GET /questions/design (random)` and `GET /questions/design/{id}` (specific). No rate limit on either.

Why it exists: Lets the frontend show the user a question to design against. The cost gate is on submission, not on fetch.

#17 `backend/app/api/routes/attempts.py`

What it is: Adds `POST /attempts/design` — the headline endpoint of the entire batch. Returns 201 Created with the full feedback bundle on success, 429 on rate limit, 502 on AI failure.

Why it exists: This is the single most important endpoint in the entire app. Everything in this batch exists to make this one POST work reliably and cheaply.

Group 8: Wiring (4 files modified)

Connecting the new pieces into the existing app.

#18 `backend/app/config/settings.py`

What it is: Adds one new setting: `feedback_cache_ttl_days` (default 30, min 1, max 365).

Why it exists: How long to keep cached feedback before MongoDB's TTL index deletes it. 30 days is the sweet spot — long enough to capture repeat submissions, short enough that prompt changes eventually flush the cache.

#19 `backend/app/config/container.py`

What it is: Wires the two new services into the dependency-injection container: the diagram hash service (as a Singleton, since it has no state) and the design feedback service (as a Factory, with all its dependencies injected).

Why it exists: This is where ports get connected to adapters. Adding new functionality here is a one-stop edit — the rest of the app sees no difference.

#20 `backend/app/main.py`

What it is: Bumps the version to 0.4.0. The startup hook still calls `ensure_indexes()`, which now creates the TTL index for the feedback cache.

Why it exists: The TTL index is what makes cache cleanup automatic. Without this hook, expired entries would accumulate forever — a slow leak.

#21 `backend/.env.example`

What it is: Documents the new `FEEDBACK_CACHE_TTL_DAYS=30` environment variable with a comment explaining when to tune it.

Why it exists: Keeps the `.env.example` file as the single source of truth for every config knob. New contributors copy it and know exactly what each setting does.

Group 9: Seed data (2 files)

#22 `backend/scripts/seed_design_questions.json`

What it is: 10 design questions: 3 junior (URL shortener, pastebin, image upload), 4 mid (Twitter timeline, WhatsApp chat, distributed rate limiter, notifications), 3 senior (Instagram feed, analytics pipeline, collaborative editor). Each has scale numbers and constraints baked into the prompt.

Why it exists: These prompts are what the user practices against. The scale numbers ("200M DAU, p99 200ms") are critical — they anchor the AI's calibration. Without them, the AI gives generic textbook answers; with them, the AI can call out specifics.

#23 `backend/scripts/seed_design_questions.py`

What it is: An idempotent loader: reads the JSON, upserts by (type, title) so re-running the script is safe.

Why it exists: First run inserts 10 questions. Second run prints "skip" for all 10. Lets you re-run safely without duplicating data.

Group 10: Frontend (1 zip = 18 new files + 3 modified)

Shipped as a single zip because the frontend feature is too tangled to review file-by-file. Same pattern as Batch 3.

File	Purpose
------	---------

<code>types.ts</code>	TypeScript types mirroring backend schemas (camelCase)
<code>schema.ts</code>	Zod parsers with snake_case → camelCase transforms
<code>api.ts</code>	API call functions
<code>palette.ts</code>	14 component definitions (icons, labels, categories)
<code>store.ts</code>	Zustand state for question, feedback, selected item
<code>hooks/useDesignQuestion.ts</code>	Fetch random question (mutation, not auto-refetching)
<code>hooks/useSubmitDesign.ts</code>	Submit design with 30s timeout
<code>hooks/useDiagramExport.ts</code>	Strip UI fields before submission
<code>components/Canvas.tsx</code>	React Flow surface with drop handler
<code>components/ComponentPalette.tsx</code>	Draggable left sidebar
<code>components/nodes/ComponentNode.tsx</code>	Single config-driven node (all 14 types)
<code>components/EmptyCanvasHint.tsx</code>	Onboarding overlay when canvas is empty
<code>components/QuestionHeader.tsx</code>	Top toolbar with prompt + submit + counts
<code>components/FeedbackPanel.tsx</code>	Right-side severity-grouped feedback
<code>components/FeedbackItemCard.tsx</code>	Clickable item that highlights canvas nodes
<code>components/ScoreSummary.tsx</code>	Score + level badge + strengths
<code>components/SubmittingOverlay.tsx</code>	Loading state during 10–30s wait
<code>DesignCanvasPage.tsx</code>	Orchestrating page (default export)

How to verify Batch 4 works

Step 1 — Apply backend files

Copy the 22 backend files from batch-4/backend/ . . . into your repo. Make sure `FEEDBACK_CACHE_TTL_DAYS=30` is in your `.env.local`.

Step 2 — Restart the backend

```
docker compose up -d
```

The startup hook creates the TTL index for the feedback cache.

Step 3 — Seed design questions

```
cd backend && python scripts/seed_design_questions.py
```

First run: 10 inserts. Second run: 10 skips. Both safe.

Step 4 — Apply frontend zip

```
unzip -o batch-4-frontend.zip && cd frontend && npm install && npm run dev
```

Step 5 — Walk through the UI

- Sign in (mock auth)
- Click "Design canvas" card on home page
- Click "Load question" — see prompt populate
- Drag 4–6 components from palette onto canvas
- Connect them by dragging from one node's edge to another's
- Type something in the notes field on the right
- Click "Submit design" — see loading overlay for 10–20 seconds
- Feedback panel slides in with score, strengths, severity-grouped gaps
- Click any feedback item — affected nodes get colored ring, others dim
- Submit the same diagram again — returns instantly with "Served from cache" indicator
- Submit a third time — see rate-limit banner (you've used today's 2)

What's NOT included (deferred)

Feature	Why deferred
Streaming feedback (SSE)	Synchronous request is simpler for MVP; 10–30s wait is OK
Edge labels	Direction is enough for the AI; labels add prompt complexity
Past attempts viewer	History page itself is in Batch 6 polish
Reference architecture sketches	User designs from scratch — stretch goal
Frontend tests for canvas	Same deferral as Batch 3 — Batch 6 polish
Vision-based feedback	Out of scope per ADR-005
Cost display per submission	Stretch; optional for portfolio polish
Diagram persistence on reload	Zustand resets on refresh — own design decision

What's next: Batch 5 — Azure migration + CI/CD

The architecture pays off. Per ADR-002, the same MongoDB code works against Cosmos with just a different connection string — no new database adapter needed. Real work in Batch 5:

- Provision Azure resources (Static Web Apps, Functions, Cosmos, Key Vault, App Insights)
- Write 4 Azure-specific adapters: real Microsoft Entra ID auth, App Insights telemetry, Key Vault secrets, no new database adapter
- Update the DI container with an "azure" branch

- GitHub Actions pipeline: lint → typecheck → test → build → deploy
- Budget alert at \$5
- ADR-007 documenting the Docker Compose choice
- First successful production deploy

To begin Batch 5, say: "start Batch 5".